

Ejercicio 1

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<String> ciudades = Arrays.asList(
            "elche",
            "alicante",
            "elda",
            "petrer",
            "aspe",
            "crevillente",
            "alcoy",
            "ibi",
            "benidorm",
            "matola"
        );
        ciudades.stream()
            .map(ciudad ->
                ciudad.substring(0,1).toUpperCase()
                + ciudad.substring(1))
            .sorted()
            .forEach(System.out::println);
    }
}
```

Ejercicio 2

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Main {

    public static void main(String[] args) {

        List<String> ciudades = Arrays.asList(

            "elche",
            "alicante",
            "elda",
            "petrer",
            "aspe",
            "crevillente",
            "alcoy",
            "ibi",
            "benidorm",
            "matola"

        );

        List<String> resultado =

            ciudades.stream()

                .map(ciudad ->

                    ciudad.substring(0,1)

                        .toUpperCase()

                    + ciudad.substring(1))

                .peek(System.out::println)

                .sorted()

                .collect(Collectors.toList());

        System.out.println(resultado);

    }
```

```
}
```

Ejercicio 3

```
import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<String> ciudades = Arrays.asList(
            "elche",
            "alicante",
            "elda",
            "petrer",
            "aspe",
            "crevillente",
            "alcoy",
            "ibi",
            "benidorm",
            "matola"
        );
        long cantidad =
            ciudades.stream()
                .map(ciudad ->
                    ciudad.substring(0,1)
                        .toUpperCase()
                    + ciudad.substring(1))
                .filter(
                    ciudad ->
                        ciudad.startsWith("A")
                )
                .count();
    }
}
```

```
        System.out.println(cantidad);
    }
}
```

Ejercicio 4

Clase Viaje

```
public class Viaje {
    private String pais;
    public Viaje(String pais) {
        this.pais = pais;
    }
    public String getPais() {
        return pais;
    }
}
```

Clase Persona

```
import java.util.ArrayList;
import java.util.List;
public class Persona {
    private String nombre;
    private List<Viaje> viajes;
    public Persona(String nombre) {
        this.nombre = nombre;
        this.viajes = new ArrayList<>();
    }
    public String getNombre() {
        return nombre;
    }
    public List<Viaje> getViajes() {
        return viajes;
    }
}
```

```
}  
}
```

Main

```
import java.util.Arrays;  
import java.util.List;  
public class Main {  
    public static void main(String[] args) {  
        Persona p1 = new Persona("Adrian");  
        Persona p2 = new Persona("Mario");  
        p1.getViajes().add(new Viaje("España"));  
        p1.getViajes().add(new Viaje("Italia"));  
        p2.getViajes().add(new Viaje("Francia"));  
        p2.getViajes().add(new Viaje("Alemania"));  
        List<Persona> personas =  
            Arrays.asList(p1, p2);  
        // Con bucles  
        for (Persona persona : personas) {  
            for (Viaje viaje :  
                persona.getViajes()) {  
                System.out.println(  
                    persona.getNombre()  
                    + " -> "  
                    + viaje.getPais()  
                );  
            }  
        }  
        // Con streams  
        personas.stream()  
            .flatMap(  
                persona -> persona.getViajes().stream(),  
                true  
            )  
            .forEach(  
                viaje -> System.out.println(  
                    viaje.getNombre() + " -> " + viaje.getPais()  
                )  
            );  
    }  
}
```

```

        persona ->
            persona.getViajes()
                .stream()
                .map(viaje ->
                    persona.getNombre()
                    + " -> "
                    + viaje.getPais())
                )
            .forEach(System.out::println);
    }
}

```

Ejercicio 5

A

```

import java.util.ArrayList;
import java.util.List;
import java.util.OptionalInt;
public class Main {
    public static void main(String[] args) {
        List<Punto> puntos =
            new ArrayList<>();
        puntos.add(new Punto(-4,-8));
        puntos.add(new Punto(-2,9));
        puntos.add(new Punto(-1,-8));
        puntos.add(new Punto(0,-7));
        puntos.add(new Punto(1,1));
        puntos.add(new Punto(2,3));
        puntos.add(new Punto(2,3));
        puntos.add(new Punto(2,-2));
        puntos.add(new Punto(4,-1));
    }
}

```

```

OptionalInt suma =
    puntos.stream()
        .mapToInt(Punto::getX)
        .reduce((a,b) -> a + b);
suma.ifPresent(System.out::println);
}
}

```

B

```

import java.util.ArrayList;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<Punto> puntos =
            new ArrayList<>();
        puntos.add(new Punto(-4,-8));
        puntos.add(new Punto(-2,9));
        puntos.add(new Punto(-1,-8));
        puntos.add(new Punto(0,-7));
        puntos.add(new Punto(1,1));
        int suma =
            puntos.stream()
                .mapToInt(Punto::getX)
                .peek(System.out::println)
                .sum();
        System.out.println(suma);
    }
}

```

C

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
public class Main {
    public static void main(String[] args) {
        List<Punto> puntos =
            new ArrayList<>();
        puntos.add(new Punto(-4,-8));
        puntos.add(new Punto(1,1));
        puntos.add(new Punto(2,3));
        puntos.add(new Punto(4,-1));
        List<Punto> positivos =
            puntos.stream()
                .filter(
                    punto ->
                        punto.getX() > 0
                )
                .collect(Collectors.toList());
        System.out.println(positivos);
    }
}
```

Ejercicio 6

A

```
System.out.println(
    puntuaciones.stream().count()
);
```


B

```
double media =  
    puntuaciones.stream()  
        .mapToInt(  
            PuntuacionJugador::getPuntuacion  
        )  
        .average()  
        .orElse(0);  
System.out.println(media);
```

C

```
long cantidad =  
    puntuaciones.stream()  
        .filter(  
            jugador ->  
                jugador.getPuntuacion() >= 90  
        )  
        .count();  
System.out.println(cantidad);
```

D

```
List<String> nombres =  
    puntuaciones.stream()  
        .filter(  
            jugador ->  
                jugador.getPuntuacion() < 70  
        )  
        .map(  
            PuntuacionJugador::getNombre  
        )  
        .collect(Collectors.toList());
```

E

```
nombres.forEach(System.out::println);
```

F

```
puntuaciones.stream()
    .sorted(
        Comparator.comparing(
            PuntuacionJugador::getApellido
        )
    )
    .forEach(
        jugador ->
            System.out.println(
                jugador.getNombre()
                + " "
                + jugador.getPuntuacion()
            )
    );
```

G

```
puntuaciones.stream()
    .sorted(
        Comparator.comparingInt(
            PuntuacionJugador::getPuntuacion
        )
    )
    .forEach(
        jugador ->
            System.out.println(
                jugador.getNombre()
                + " "
```

```

        + jugador.getPuntuacion()
    )
);

```

H

```

PuntuacionJugador ultimo =
    puntuaciones.stream()
        .reduce((a,b) -> b)
        .orElse(null);
System.out.println(
    ultimo.getNombre()
);

```

Ejercicio 7

```

import java.util.Arrays;
import java.util.List;
public class Main {
    public static void main(String[] args) {
        List<List<String>> array =
            Arrays.asList(
                Arrays.asList("a","b"),
                Arrays.asList("c","d"),
                Arrays.asList("e","f")
            );
        array.stream()
            .flatMap(List::stream)
            .filter(
                letra ->
                    !letra.equals("a")
            )
            .forEach(System.out::println);
    }
}

```

```
}  
}
```

Ejercicio 8

A

```
List<Producto> resultado =  
    productos.stream()  
        .filter(  
            producto ->  
                producto.getCategoria()  
                    .equals("IT")  
                &&  
                producto.getPrecio() > 50  
        )  
        .collect(Collectors.toList());
```

B

```
pedidos.stream()  
    .filter(  
        pedido ->  
            pedido.getProductos()  
                .stream()  
                .anyMatch(  
                    producto ->  
                        producto.getCategoria()  
                            .equals("Ropa")  
                )  
    )  
    .forEach(System.out::println);
```

C

```
List<Producto> juguetes =
    productos.stream()
        .filter(
            producto ->
                producto.getCategoria()
                    .equals("Juguetes")
        )
        .map(producto -> {

            producto.setPrecio(
                producto.getPrecio() * 0.9
            );
            return producto;
        })
        .collect(Collectors.toList());
```

D

```
List<Pedido> resultado =
    pedidos.stream()
        .filter(
            pedido ->
                pedido.getFechaPedido()
                    .isAfter(
                        LocalDate.of(
                            2022,1,1)
                    )
        )
        .collect(Collectors.toList());
```

E

```
Producto barato =  
    productos.stream()  
        .filter(  
            producto ->  
                producto.getCategoria()  
                    .equals("Libros")  
        )  
        .min(  
            Comparator.comparingDouble(  
                Producto::getPrecio  
            )  
        )  
        .orElse(null);
```

F

```
List<Producto> productosHoy =  
    pedidos.stream()  
        .filter(  
            pedido ->  
                pedido.getFechaPedido()  
                    .equals(  
                        LocalDate.now()  
                    )  
        )  
        .peek(System.out::println)  
        .flatMap(  
            pedido ->  
                pedido.getProductos()  
                    .stream()  
        )
```

```
)  
.collect(Collectors.toList());
```

G

```
double total =  
    pedidos.stream()  
        .flatMap(  
            pedido ->  
                pedido.getProductos()  
                    .stream()  
        )  
        .mapToDouble(  
            Producto::getPrecio  
        )  
        .sum();  
System.out.println(total);
```